Encontro 7: Ferramentas e Ecossistema (Vite & Vue Router)

Sumário

Encontro 7: Ferramentas e Ecossistema (Vite & Vue Router) 1

1. Introdução 2
   1.1. Vantagens de Uso 2
   1.2. Exemplos de Aplicações Práticas 2
2. Vite: O Futuro das Ferramentas de Build 3
   2.1. Como o Vite é tão rápido? 3
   2.2. Criando um Projeto com Vite 3
3. Vue Router: Navegação em Single-Page Applications (SPAs) 4
   3.1. Conceitos Fundamentais 4
4. Estrutura Modular de Rotas 5
5. Exercício Prático Integrado: Criando um Mini Blog 6

1. Introdução

À medida que as aplicações Vue.js se tornam mais complexas e dinâmicas, surge a necessidade de ferramentas modernas que facilitem o desenvolvimento, a organização e o desempenho do projeto. Nesse contexto, destacam-se duas tecnologias essenciais do ecossistema Vue atual: **Vite** e **Vue Router**.

O **Vite** é uma ferramenta de build de última geração, criada para oferecer velocidade e eficiência no desenvolvimento front-end. Ele utiliza o poder dos módulos nativos do ES (ES Modules) e um servidor de desenvolvimento ultrarrápido, permitindo que o desenvolvedor veja as mudanças no código quase instantaneamente. Além disso, o Vite otimiza a aplicação automaticamente para produção, gerando pacotes leves e com excelente desempenho.

Já o **Vue Router** é a biblioteca oficial de roteamento do Vue.js, responsável por gerenciar a navegação entre páginas e componentes em uma Single-Page Application (SPA). Ele possibilita que a aplicação tenha múltiplas "páginas" sem recarregar o navegador, tornando a navegação fluida e rápida, além de permitir recursos avançados como rotas dinâmicas, proteção de rotas (guardas) e transições animadas entre telas.

1.1. Vantagens de Uso

- Desempenho e agilidade: o Vite oferece inicialização quase instantânea e recarregamento rápido durante o desenvolvimento.
- Modularidade e organização: o Vue Router ajuda a estruturar melhor a aplicação, separando lógicas de navegação em rotas claras e bem definidas.
- Produtividade: ambos simplificam tarefas comuns, reduzindo o tempo gasto com configurações complexas.
- Experiência do usuário aprimorada: com o Vue Router, a navegação ocorre sem recarregar a página, proporcionando uma experiência fluida e profissional.

1.2. Exemplos de Aplicações Práticas

- Painéis administrativos (dashboards) que utilizam múltiplas rotas para diferentes seções, como usuários, relatórios e configurações.
- Sistemas educacionais ou e-commerces, onde o Vue Router gerencia páginas como "Home", "Produtos" e "Carrinho".
- Aplicações de grande porte desenvolvidas com Vite, aproveitando sua rapidez no ambiente de desenvolvimento e sua capacidade de gerar builds otimizadas para produção.

Em resumo, o Vite garante velocidade, eficiência e simplicidade no desenvolvimento. O Vue Router proporciona organização e navegação fluida — juntos, tornam possível construir aplicações Vue.js modernas, escaláveis e de alto desempenho.

2. Vite: O Futuro das Ferramentas de Build

Para desenvolver páginas Vue sem usar Docker, costumamos usar o Vue CLI, que por baixo dos panos utiliza o Webpack. O Vite ("rápido" em francês, pronuncia-se "vit") é uma alternativa criada por Evan You (criador do Vue) que permite desenvolver de forma drasticamente mais rápida.

2.1. Como o Vite é tão rápido?

- **Vue CLI (Webpack)**: Antes de você ver sua aplicação, ele empacota (bundles) todo o seu código JavaScript em um ou mais arquivos. Em aplicações grandes, esse processo pode levar vários segundos ou até minutos.
- **Vite**: Utiliza os módulos ES nativos do navegador. Em vez de empacotar tudo, ele serve os arquivos sob demanda diretamente no navegador. O tempo de inicialização do servidor de desenvolvimento é quase instantâneo e Hot Module Replacement (HMR) extremamente rápido.

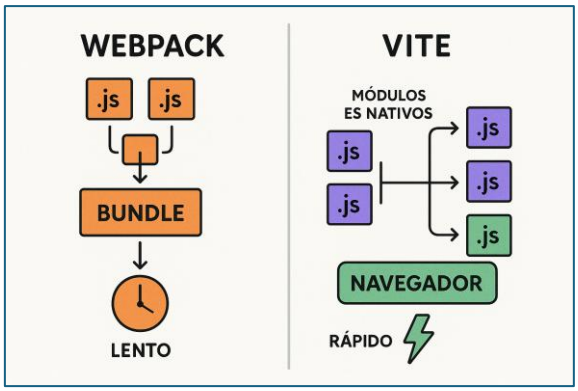


Figura 1: Comparação simplificada do processo de build.

Na figura 1, vemos como o Vite serve os módulos nativamente, enquanto o Webpack precisa empacotar todo o código antes de servir.

2.2. Criando um Projeto com Vite

Para criar um novo projeto Vue com Vite, o comando é simples e interativo:

```
npm init @vitejs/app
```

O terminal irá guiá-lo por uma série de perguntas para configurar seu projeto, incluindo a adição do Vue Router, Pinia, ESLint, etc.

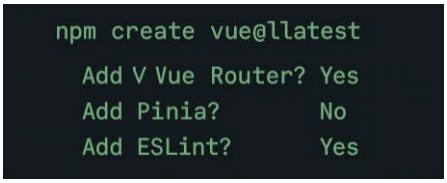


Figura 2: Processo de criação de projeto Vue.

A figura 2 ilustra o processo de criação de um projeto com `npm init @vitejs/app`, durante o qual você pode customizar as ferramentas incluídas.

3. Vue Router: Navegação em Single-Page Applications (SPAs)

Uma SPA é uma aplicação web que carrega uma única página HTML e atualiza seu conteúdo dinamicamente conforme o usuário interage com ela, sem recarregar a página inteira. Para gerenciar a "navegação" entre diferentes "páginas" (que na verdade são componentes), usamos um roteador.

O **Vue Router** é a biblioteca oficial para isso. Ele mapeia URLs do navegador para componentes Vue específicos.

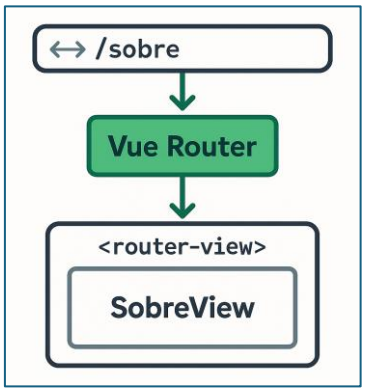


Figura 3: Funcionamento do Vue Router

A figura 3 mostra que o Vue Router intercepta a URL do navegador e renderiza o componente correspondente dentro da tag `<router-view>`.

3.1. Conceitos Fundamentais

a) `createRouter`: Função para criar a instância do roteador.
b) `createWebHistory`: Modo de histórico que usa a History API do HTML5 para navegação sem o `#` na URL (ex: `/sobre` em vez de `#sobre`).
c) `routes`: Um array de objetos onde cada objeto define uma rota (um mapeamento entre um caminho, um nome e um componente).
d) `<router-link>`: Componente para criar links de navegação. Ele é renderizado como uma tag `<a>`, mas previne o recarregamento da página.
e) `<router-view>`: Componente que atua como um marcador de posição onde o componente da rota ativa será renderizado.

4. Estrutura Modular de Rotas

Em um projeto criado com Vite e Vue Router, a configuração das rotas é, por padrão, modular. Em vez de ter tudo no `main.js`, temos uma estrutura dedicada:

- `src/router/index.js`: Arquivo central que define e exporta a instância do roteador.
- `src/views/`: Diretório que contém os componentes que representam páginas inteiras (ex: `HomeView.vue`, `AboutView.vue`).

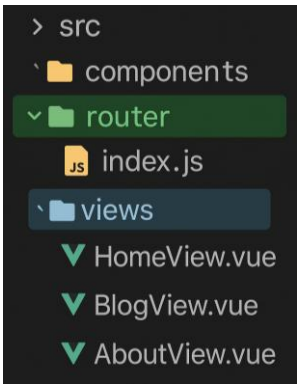


Figura 4: Estrutura de pastas recomendada para um projeto com Vue Router

Na figura 4 vemos que a estrutura de pasta recomendada para um projeto com Vue Router separa as "views" (páginas) e a configuração do roteador.

5. Exercício Prático Integrado: Criando um Mini Blog

Objetivo: Construir uma pequena SPA com três páginas (Home, Blog, e Detalhe do Post) usando Vite e Vue Router.

Passo 1: Criar o Projeto

a) Num prompt do Windows, execute `npm init @vitejs/app`, nomeie o projeto como `exercicio7` e selecione "`Yes`" para adicionar o Vue Router.
b) Em "`Select a framework`", use a seta para ir até "Vue" e tecle `<ENTER>`.
c) Em "`Select a variant`", use a seta para ir até "Official Vue Starter" e tecle `<ENTER>`.
d) Em "`Use rolldown-vite (Experimental)`" apenas tecle `<ENTER>`.
e) Em "`Install with npm and start now`" apenas tecle `<ENTER>`.
f) Em "`Select features to include in your project`", use a seta para ir até "`Router (SPA development)`", clique na barra de espaço para marcar.
g) Em "`Select experimental features to include in your project`" apenas tecle `<ENTER>`.
h) Em "`Skip all example code and start with a blank Vue project`" apenas tecle `<ENTER>`.
i) Vá para o diretório exercicio7.
j) Execute `npm i`
k) Execute `npm run dev`

Passo 2: Criar as Views

Dentro de `src/views`, crie os seguintes arquivos:

- `HomeView.vue`: Uma página de boas-vindas simples.
- `BlogView.vue`: Listará os posts.
- `PostDetailView.vue`: Mostrará o conteúdo de um post específico.

Passo 3: Configurar as Rotas (`src/router/index.js`)

Substitua o conteúdo do arquivo pelo código abaixo. Note a rota dinâmica `/post/:id`.

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import BlogView from '../views/BlogView.vue'
import PostDetailView from '../views/PostDetailView.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: HomeView
    },
    {
```

```
      path: '/blog',
      name: 'blog',
      component: BlogView
    },
    {
      path: '/post/:id',
      name: 'postDetail',
      component: PostDetailView
    }
  ]
})
export default router
```

Passo 4: Atualizar o Layout (App.vue)

Adicione a navegação principal e o `<router-view>`.

```
<template>
  <header>
    <nav>
      <router-link to="/">Home</router-link>
      <router-link to="/blog">Blog</router-link>
    </nav>
  </header>
  <main>
    <router-view />
  </main>
</template>

<style scoped>
header { background: #f2f2f2; padding: 15px; text-align: center; }
nav a {
  font-weight: bold;
  color: #2c3e50;
  margin: 0 15px;
  text-decoration: none;
}
nav a.router-link-exact-active { color: #42b983; }
main { padding: 20px; }
</style>
```

Passo 5: Implementar as Views

`HomeView.vue`

```
<template>
  <div class="home">
    <h1>Bem-vindo ao Mini Blog!</h1>
    <p>Navegue até a seção de Blog para ver os posts.</p>
  </div>
</template>
```

`BlogView.vue` (com links dinâmicos)

```
<template>
  <div class="blog">
    <h1>Posts Recentes</h1>
    <ul>
      <li v-for="post in posts" :key="post.id">
        <router-link :to="{ name: 'postDetail', params: { id: post.id } }">
          {{ post.title }}
        </router-link>
      </li>
    </ul>
  </div>
</template>

<script>
export default {
  data() {
    return {
      posts: [
        { id: 1, title: 'O que é Vue.js?' },
        { id: 2, title: 'Entendendo Componentes' },
        { id: 3, title: 'A Mágica do Vue Router' }
      ]
    };
  }
}
</script>
```

`PostDetailView.vue` (acessando parâmetros da rota)

```
<template>
  <div class="post-detail">
    <h1>Exibindo o Post ID: {{ postId }}</h1>
    <p>Aqui seria exibido o conteúdo completo do post.</p>
    <router-link to="/blog">Voltar para a lista</router-link>
  </div>
```

```
</template>

<script>
export default {
  computed: {
    postId() {
      return this.$route.params.id;
    }
  }
}
</script>
```

Resultado Final

Execute `npm run dev` e navegue pela sua nova SPA! Você poderá ir da Home para o Blog, e de lá para os detalhes de cada post, tudo sem recarregar a página.

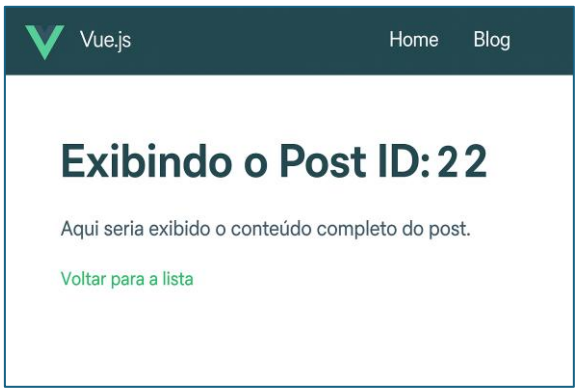


Figura 5: A página de detalhe do post, mostrando o ID dinâmico capturado da URL.